PropertySupport AI

Tutoriel d'installation, d'execution et de validation

FastAPI 8001 - Langfuse 3001 - Ollama 11434

Ce document est le tutoriel officiel du livrable PropertySupport AI. Il explique comment installer la stack, configurer l'environnement local, lancer les services, tester les endpoints et valider les principaux cas fonctionnels du projet.

Brique	**Role**
FastAPI	API HTTP
LangGraph	Orchestration du workflow metier
LangChain	Integration LLM et prompts
Ollama	Execution locale du modele LLM
SQLAlchemy 2.x	ORM
SQLite	Base locale
Alembic	Migrations DB
Langfuse	Observabilite GenAI
Pydantic	Validation DTO et sorties LLM
pytest / Ruff / Pyright	Tests, lint, type checking
Docker	Conteneurisation
Service	**URL locale**
FastAPI	http://localhost:8001
Swagger UI	http://localhost:8001/docs
ReDoc	http://localhost:8001/redoc
Langfuse	http://localhost:3001
Ollama	http://localhost:11434

1. Environnement cible

Element	**Recommandation**
OS	Windows 10 / 11
Shell	PowerShell
Editeur	Visual Studio Code 2026
Python	3.11+
LLM local	Ollama
Modele recommande	mistral-nemo:12b
Base de donnees	SQLite

Observabilite	Langfuse
Port FastAPI	8001
Port Langfuse	3001
Docker	Docker Desktop

Le tutoriel est volontairement oriente Windows + PowerShell.

2. Installer les prerequis systeme

2.1 Installer Python 3.11+

Telecharger Python depuis le site officiel :

https://www.python.org/downloads/

Pendant l installation, cocher :

Add python.exe to PATH

Verifier ensuite dans PowerShell :

python --version

Resultat attendu :

Python 3.11.x

2.2 Installer Git

https://git-scm.com/download/win

git --version

2.3 Installer Visual Studio Code 2026

https://code.visualstudio.com/

code --version

Si la commande code n est pas reconnue, ouvrir VS Code puis activer la commande depuis la palette :

Shell Command: Install 'code' command in PATH

2.4 Installer Docker Desktop

https://www.docker.com/products/docker-desktop/

docker --version

docker compose version

2.5 Installer Ollama

https://ollama.com/

ollama --version

ollama serve

Si le port est deja utilise, c est probablement que le service Ollama tourne deja en arriere-plan.

2.6 Installer uv

uv cree le venv et installe les dependances Python du projet.

```
irm https://astral.sh/uv/install.ps1 | iex
```

Fermer puis rouvrir PowerShell, puis verifier :

```
uv --version
```

3. Installer le modele LLM local

```
ollama pull mistral-nemo:12b
```

Modele optionnel plus leger :

```
ollama pull qwen2.5-coder:7b
```

```
ollama list
```

```
ollama run mistral-nemo:12b
```

```
Reply with a short JSON object containing a hello field.
/bye
```

4. Installer les extensions VS Code

```
code --install-extension ms-python.python
code --install-extension ms-python.vscode-pylance
code --install-extension charliermarsh.ruff
code --install-extension ms-azuretools.vscode-docker
code --install-extension humao.rest-client
code --install-extension qwtel.sqlite-viewer
code --install-extension redhat.vscode-yaml
code --install-extension tamasfe.even-better-toml
code --install-extension editorconfig.editorconfig
```

Extension optionnelle LangGraph Visualizer :

```
code --install-extension smazee.langgraph-visualizer
```

Si cette extension n est pas disponible, le graphe reste consultable dans app/application/graph.py et dans Langfuse via le graph_path.

5. Ouvrir le projet

```
cd C:\Users\damie\Desktop\Dev\PropertySupportAI\property-support-genai-langfuse-ports-patched\property-support-genai
```

```
code .
```

La racine du projet doit contenir README.md, pyproject.toml, alembic.ini, .env.example, app/, scripts/, docs/, tests/ et docker-compose.yml.

6. Debloquer les fichiers extraits du ZIP

```
Get-ChildItem -Recurse | Unblock-File
```

7. Autoriser les scripts PowerShell

Autorisation temporaire dans le terminal courant :

```
Set-ExecutionPolicy -Scope Process Bypass
```

Autorisation durable pour les scripts locaux de l utilisateur :

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

8. Creer et configurer .env

```
Copy-Item .env.example .env
code .env
```

```
APP_ENV=local
APP_DEBUG=true
APP_VERSION=0.1.0
APP_PORT=8001

DATABASE_URL=sqlite:///./property_support.db
DATABASE_ECHO=false

SAFE_MODE=true
WRITE_TICKET_EVENTS=true
DRY_RUN=false

LLM_PROVIDER=ollama
LLM_BASE_URL=http://localhost:11434
LLM_MODEL=mistral-nemo:12b
LLM_TIMEOUT_SECONDS=90
LLM_MAX_RETRIES=1

REQUEST_TIMEOUT_SECONDS=120
DATABASE_TIMEOUT_SECONDS=10

LLM_TEMPERATURE_CLASSIFICATION=0.0
LLM_TEMPERATURE_EXTRACTION=0.0
LLM_TEMPERATURE_REASONING=0.1
LLM_TEMPERATURE_WRITING=0.3

LANGFUSE_ENABLED=false
LANGFUSE_HOST=http://localhost:3001
LANGFUSE_BASE_URL=http://localhost:3001
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=

LOG_LEVEL=INFO
LOG_FORMAT=human

API_MAX_MESSAGE_LENGTH=4000
INCLUDE_DEBUG_METADATA=true
INCLUDE_GRAPH_PATH=true
MIN_CLASSIFICATION_CONFIDENCE=0.60
```

```
DUPLICATE_TICKET_WINDOW_HOURS=72
MAX_RECENT_TICKETS=20
```

9. Installer les dependances Python

```
.\scripts\install.ps1
```

Si PowerShell bloque encore :

```
powershell.exe -ExecutionPolicy Bypass -File .\scripts\install.ps1
```

Verifications utiles :

```
uv run python -c "import sqlalchemy; print('SQLAlchemy OK', sqlalchemy.__version__)"
```

```
uv run python -c "import fastapi; print('FastAPI OK', fastapi.__version__)"
```

```
uv run python -m alembic --version
```

10. Selectionner l interpreteur Python dans VS Code

1. Ctrl+Shift+P
2. Python: Select Interpreter
3. Selectionner .venv\Scripts\python.exe

11. Lancer les migrations

```
.\scripts\migrate.ps1
```

Equivalent direct :

```
uv run python -m alembic upgrade head
```

```
uv run python -m alembic current
```

12. Seeder la base SQLite

```
.\scripts\seed.ps1 -Reset
```

Equivalent direct :

```
uv run python -m scripts.seed_db --reset
```

La base locale creee est property_support.db. Regle centrale :

```
tickets       = etat courant, lecture seule pour le workflow GenAI
ticket_events = suggestions IA et journal safe-mode
```

13. Inspecter la base SQLite

Avec SQLite Viewer, ouvrir property_support.db. Tables utiles : buildings, providers, contracts, tickets, ticket_events, procedures.

```
sqlite3 property_support.db
```

```
.tables
SELECT id, name, city FROM buildings;
SELECT id, title, status FROM tickets;
```

```
SELECT id, event_type, created_at FROM ticket_events;
.exit
```

14. Demarrer Ollama

```
ollama serve
```

```
ollama list
```

15. Demarrer FastAPI sur le port 8001

```
.\scripts\dev.ps1
```

Equivalent direct :

```
uv run python -m uvicorn app.main:app --reload --host 127.0.0.1 --port 8001
```

```
Uvicorn running on http://127.0.0.1:8001
```

16. Ouvrir la documentation FastAPI

```
Swagger UI : http://localhost:8001/docs
ReDoc      : http://localhost:8001/redoc
```

```
GET  /health
GET  /version
GET  /data-health
GET  /demo/examples
POST /ask
GET  /ticket-events
GET  /ticket-events/{event_id}
```

17. Verifier que la bonne API repond

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/health" `
  -Method Get | ConvertTo-Json -Depth 10
```

```
$openapi = Invoke-RestMethod -Uri "http://localhost:8001/openapi.json" -Method Get
$openapi.paths.PSObject.Properties.Name
```

```
/health
/version
/data-health
/demo/examples
/ask
/ticket-events
```

18. Verifier la sante des donnees

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/data-health" `
  -Method Get | ConvertTo-Json -Depth 10
```

```
{
  "status": "ok",
  "tables": {
    "buildings": 3,
    "providers": 6,
    "contracts": 5,
    "procedures": 3,
    "tickets": 3
  },
  "errors": [],
  "prompts": "ok",
  "graph": "compiled"
}
```


19. Configurer l affichage UTF-8 dans PowerShell

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
$OutputEncoding = [System.Text.Encoding]::UTF8
chcp 65001
```

Si les accents sont corrects dans Swagger mais pas dans PowerShell, l API est correcte : c est l affichage console qui est en cause.

20. Requetes PowerShell de validation

20.1 Healthcheck

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/health" `
  -Method Get | ConvertTo-Json -Depth 10
```

20.2 Version

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/version" `
  -Method Get | ConvertTo-Json -Depth 10
```

20.3 Data health

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/data-health" `
  -Method Get | ConvertTo-Json -Depth 10
```

20.4 Demo examples

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/demo/examples" `
  -Method Get | ConvertTo-Json -Depth 10
```

20.5 Lookup - prestataire chauffage avec nom complet

```
$body = @{
  message = "Qui est le prestataire chauffage de la Résidence Les Érables ?"
} | ConvertTo-Json -Depth 10
```

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

Attendu : intent=lookup, status=success, sources includes buildings:B001, contracts:C001, providers:P001

20.6 Lookup - prestataire chauffage avec nom court

```
$body = @{
  message = "Qui est le prestataire chauffage des Érables ?"
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

20.7 Lookup - tickets ouverts

```
$body = @{
  message = "Quels tickets sont ouverts pour la Résidence Les Érables ?"
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

20.8 Incident analysis - chauffage collectif

```
$body = @{
  message = "Dans la Résidence Les Érables, plusieurs résidents n’ont plus de chauffage. Que dois-je faire ?"
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

20.9 Communication - intervention chauffage

```
$body = @{
  message = "Rédige un message aux résidents de la Résidence Les Érables pour les informer qu'une intervention chauffage
est prévue demain."
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

20.10 Ticket workflow - ajout a un ticket existant

```
$body = @{
  message = "Ajoute au ticket chauffage des Érables que trois résidents supplémentaires ont signalé la panne."
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

Attendu : status=ticket_event_created, ticket_id=T001, event_type=ai_update_suggested, safe_mode=true

20.11 Verifier les ticket events

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/ticket-events" `
  -Method Get | ConvertTo-Json -Depth 20
```

20.12 Lire un ticket event precis

```
Invoke-RestMethod `
  -Uri "http://localhost:8001/ticket-events/E-XXXXXXXXXX" `
  -Method Get | ConvertTo-Json -Depth 20
```

20.13 Creation de ticket - fuite sans contrat actif

```
$body = @{
  message = "Crée un ticket pour une fuite signalée au plafond dans la Résidence Saint-Victor."
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

20.14 Incident - fuite d eau sans contrat actif

```
$body = @{
  message = "Mme Durand signale une fuite d’eau au plafond dans la Résidence Saint-Victor. Que dois-je faire ?"
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

20.15 Urgence - ascenseur avec personne bloquee

```
$body = @{
  message = "L’ascenseur de la Résidence Belvédère est bloqué avec une personne dedans. Que faut-il faire ?"
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
```

```
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

Attendu : intent=incident_analysis, status=success, incident_action=emergency_escalation, priority=critical, provider=LiftSecure

20.16 Clarification - demande trop vague

```
$body = @{
  message = "Il y a un problème dans l'immeuble."
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

21. Tester Langfuse

21.1 Lancer Langfuse

```
docker compose up -d
```

```
docker compose ps
```

Langfuse : http://localhost:3001

21.2 Creer les cles Langfuse

Ouvrir http://localhost:3001, creer un compte, une organisation et un projet, puis recuperer Public key et Secret key.

```
LANGFUSE_ENABLED=true
LANGFUSE_HOST=http://localhost:3001
LANGFUSE_BASE_URL=http://localhost:3001
LANGFUSE_PUBLIC_KEY=pk-lf-...
LANGFUSE_SECRET_KEY=sk-lf-...
```

```
.\scripts\dev.ps1
```

21.3 Test manuel Langfuse

```
@'
from langfuse import get_client
from app.config import settings
import os

os.environ["LANGFUSE_PUBLIC_KEY"] = settings.langfuse_public_key or ""
os.environ["LANGFUSE_SECRET_KEY"] = settings.langfuse_secret_key or ""
os.environ["LANGFUSE_BASE_URL"] = (
    settings.langfuse_base_url
    or settings.langfuse_host
    or "http://localhost:3001"
)
```

```
langfuse = get_client()

with langfuse.start_as_current_observation(
    name="manual-test-trace",
    as_type="span",
    input={"message": "hello from PropertySupport AI"},
    metadata={"source": "manual-test"},
) as observation:
    observation.update(output={"status": "ok"})

langfuse.flush()
print("Manual Langfuse trace sent")
'@ | Set-Content .          mp_langfuse_test.py -Encoding UTF8

uv run python .            mp_langfuse_test.py
Remove-Item .mp_langfuse_test.py
```

21.4 Verifier qu un appel /ask cree une trace

```
$body = @{
  message = "Qui est le prestataire chauffage des Érables ?"
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body | ConvertTo-Json -Depth 20
```

Ouvrir ensuite http://localhost:3001 et chercher property-support-ai-request.

22. Lancer les tests

```
.\scripts\test.ps1
```

Ou directement :

```
uv run pytest
```

```
uv run pytest --cov=app --cov-report=term-missing
```

23. Lancer les controles qualite

```
.\scripts\lint.ps1
```

```
uv run ruff check .
```

```
.\scripts\format.ps1
```

```
uv run ruff format .
```

```
.\scripts\typecheck.ps1
```

```
uv run pyright
```

```
.\scripts\quality.ps1
```

24. Reinitialiser la base locale

```
Remove-Item .\property_support.db -ErrorAction SilentlyContinue
.\scripts\migrate.ps1
.\scripts\seed.ps1 -Reset

Invoke-RestMethod `
  -Uri "http://localhost:8001/data-health" `
  -Method Get | ConvertTo-Json -Depth 10
```

25. Construire et lancer Docker

```
.\scripts\docker-build.ps1

docker build -t property-support-genai:latest .

docker run -d --rm `
  -p 8001:8001 `
  --name property-support-genai `
  --env-file .env `
  property-support-genai:latest

docker logs -f property-support-genai

docker stop property-support-genai
```

Swagger : http://localhost:8001/docs

26. Sequence de validation fonctionnelle

4. Lancer Ollama.
5. Lancer FastAPI sur 8001.
6. Ouvrir Swagger : http://localhost:8001/docs.
7. Appeler /data-health.
8. Tester un scenario lookup.
9. Tester un scenario communication.
10. Tester un scenario ticket_workflow.
11. Afficher /ticket-events.
12. Tester une urgence ascenseur.
13. Ouvrir Langfuse sur http://localhost:3001.
14. Verifier la trace property-support-ai-request.

```
This is an LLM-assisted workflow, not an autonomous agent.
LangGraph controls the workflow.
The LLM helps classify, extract and draft.
Business decisions remain deterministic and safe.
```

27. Depannage courant

27.1 PowerShell bloque les scripts

```
Get-ChildItem -Recurse | Unblock-File
Set-ExecutionPolicy -Scope Process Bypass
```

27.2 uv n est pas reconnu

```
irm https://astral.sh/uv/install.ps1 | iex
```

27.3 Ollama n est pas joignable

```
ollama serve
```

```
LLM_BASE_URL=http://localhost:11434
```

27.4 Le port 8001 est utilise

```
uv run python -m uvicorn app.main:app --reload --port 8002
```

27.5 Langfuse n est pas accessible

```
docker compose ps
docker compose logs langfuse-web --tail=100
```

```
Langfuse : http://localhost:3001
```

27.6 /ask retourne une erreur de body

```
$body = @{
  message = "Qui est le prestataire chauffage des Érables ?"
} | ConvertTo-Json -Depth 10

Invoke-RestMethod `
  -Uri "http://localhost:8001/ask" `
  -Method Post `
  -ContentType "application/json; charset=utf-8" `
  -Body $body
```

27.7 Accents casses dans PowerShell

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
$OutputEncoding = [System.Text.Encoding]::UTF8
chcp 65001
```

28. Commandes essentielles

Action	Commande
Installation	.\scripts\install.ps1
Migration	.\scripts\migrate.ps1
Seed	.\scripts\seed.ps1 -Reset
Run FastAPI	.\scripts\dev.ps1
Swagger	http://localhost:8001/docs
Langfuse	http://localhost:3001
Tests	.\scripts\test.ps1
Qualite	.\scripts\quality.ps1

29. Modele mental final

FastAPI on port 8001
-> Pydantic request validation
-> LangGraph workflow
-> SQLAlchemy repositories
-> SQLite business context
-> controlled LangChain / LLM calls
-> Pydantic response validation
-> safe-mode ticket events
-> Langfuse observability on port 3001

LLM-assisted workflow, not autonomous agent.

The GenAI workflow never directly modifies tickets.
It only writes human-validation suggestions to ticket_events.

Cette architecture rend le livrable robuste : le LLM aide a comprendre, extraire et rediger, mais le workflow, les regles metier, la validation et la securite restent controles par le code.